MANIPAL UNIVERSITY JAIPUR

Centre for Distance and Online Education

INTERNAL ASSIGNMENT

Course Code & Name: DCA2105 – Computer Organisation and Architecture
Program: Bachelor of Computer Applications (BCA)
Semester: III
Session: July/September 2025

Name: Hirdyansh Varshney
Roll No.: 2414500172
Course: BCA
Subject: Computer Organisation and Architecture
Assignment: Set–I & Set–II
Submission Date: 02-11-2025

Submitted To:
Manipal University Jaipur
Centre for Distance and Online Education

Set – 1

Question 1:

Answer:

Block Diagram of a Computer System

A computer system is a structured combination of hardware and software components that work together to perform data processing tasks. The block diagram of a computer system provides a simplified visual representation of its core functional units and how they interact. The primary components include the **Input Unit**, **Central Processing Unit (CPU)**, **Memory Unit**, and **Output Unit**.

At the heart of the system lies the **CPU**, often referred to as the brain of the computer. It is responsible for executing instructions and managing data flow between different units. The CPU itself is divided into two main parts: the **Arithmetic Logic Unit (ALU)** and the **Control Unit (CU)**.

The **Input Unit** allows users to enter data and instructions into the computer. Devices like keyboards, mice, and scanners fall under this category. The data collected is then sent to the memory or directly to the CPU for processing.

The **Memory Unit** stores data and instructions temporarily or permanently. It includes **Primary Memory** (RAM and ROM) for immediate access and **Secondary Storage** (like hard drives) for long-term data retention.

The **Output Unit** presents the processed data to the user through devices such as monitors, printers, or speakers.

Role of Each Unit in Detail

1. **Input Unit**: Converts user input into machine-readable form and sends it to the system for processing. It acts as a communication bridge between the user and the computer.
2. **Memory Unit**: Stores instructions, intermediate results, and final output. RAM provides temporary storage for active processes, while ROM holds essential startup instructions.
3. **Arithmetic Logic Unit (ALU)**: Performs all arithmetic operations (addition, subtraction, etc.) and logical comparisons (AND, OR, NOT). It is crucial for decision-making and calculations.
4. **Control Unit (CU)**: Directs the operation of the processor. It interprets instructions from memory and signals other components to perform tasks accordingly.
5. **Output Unit**: Translates processed data into human-readable form. It ensures users receive the results of computations in a usable format.

In summary, the block diagram of a computer system illustrates a well-coordinated architecture where each unit plays a specific role to ensure smooth and efficient data processing.

Question 2:

Answer:

Types of Micro-Operations in a Computer System

Micro-operations are the fundamental operations performed on the data stored in registers. These operations are executed during the instruction cycle and are categorized based on the type of function they perform. The four main types of micro-operations are:

1. **Register Transfer Micro-Operations**: These involve the transfer of data from one register to another. For example, moving data from register A to register B is a simple register transfer operation.
2. **Arithmetic Micro-Operations**: These perform basic arithmetic functions such as addition, subtraction, increment, decrement, and two’s complement. They are essential for executing mathematical computations.
3. **Logic Micro-Operations**: These operations manipulate individual bits of data using logical functions like AND, OR, NOT, and XOR. They are used in decision-making and bitwise data processing.

4. **Shift Micro-Operations**: These operations shift the bits of a register to the left or right. They are used in multiplication, division, and data alignment tasks.

Each of these micro-operations plays a crucial role in the execution of instructions and the overall functionality of the CPU.

Hardware Implementation of Arithmetic Micro-Operations

Arithmetic micro-operations are implemented in hardware using combinational logic circuits, primarily within the Arithmetic Logic Unit (ALU). The ALU is designed to perform various arithmetic functions based on control signals.

For example, to perform **addition**, the ALU uses a binary adder circuit, typically a ripple-carry or carry-lookahead adder. Two operands are fed into the adder, and the result is stored in a destination register.

For **subtraction**, the ALU uses the two's complement method. It inverts the bits of the subtrahend and adds one, then performs addition with the minuend.

Increment and **decrement** operations are simpler and can be implemented using half-adders or dedicated incrementer circuits.

The control unit sends specific control signals to the ALU to select the desired operation. Multiplexers are used to route the correct inputs and outputs. The result of the operation is then stored in a register or memory location.

In summary, arithmetic micro-operations are executed through well-designed hardware circuits that ensure fast and accurate computation, forming the backbone of all processing tasks in a computer.

Question 3:

Answer:

Concept of Logic Micro-Operations

Logic micro-operations are fundamental operations performed on binary data using logical functions. These operations are executed at the register level and are essential for decision-making, bit manipulation, and control flow in digital systems. Unlike arithmetic micro-operations, which deal with numerical calculations, logic micro-operations focus on bitwise transformations.

The most common logic micro-operations include:

- **AND**: Produces a 1 only if both corresponding bits are 1.
- **OR**: Produces a 1 if at least one of the corresponding bits is 1.
- **NOT**: Inverts each bit (0 becomes 1, and 1 becomes 0).
- **XOR**: Produces a 1 only if the corresponding bits are different.

These operations are used in masking, setting, clearing, and toggling specific bits in registers. For example, using AND with a mask can clear selected bits, while OR can set them. XOR is often used in parity checks and encryption algorithms.

Hardware Implementation of AND, OR, NOT, and XOR

In hardware, logic micro-operations are implemented using **logic gates** within the Arithmetic Logic Unit (ALU). Each gate performs a specific function based on its input signals.

- **AND Gate**: Takes two binary inputs and outputs 1 only if both inputs are 1. Multiple AND gates are used in parallel to perform bitwise AND on entire registers.
- **OR Gate**: Outputs 1 if at least one input is 1. Like AND gates, OR gates are arranged in parallel for register-wide operations.
- **NOT Gate (Inverter)**: Takes a single input and flips its value. It's used to implement bitwise NOT operations across all bits of a register.
- **XOR Gate**: Outputs 1 if the inputs differ. XOR gates are crucial in operations like bit toggling and parity generation.

These gates are connected to the control unit, which sends signals to select the desired operation. Multiplexers may be used to route the correct inputs to the ALU, and the result is stored in a destination register.

In summary, logic micro-operations are vital for bit-level data manipulation and are efficiently executed using dedicated hardware circuits in the ALU.

Set – 2

Question 4:

Answer:

Phases of the Instruction Cycle

The instruction cycle is the fundamental process through which a computer executes a program. It consists of four main phases: **Fetch**, **Decode**, **Execute**, and **Interrupt**. These phases are repeated continuously by the CPU to process instructions stored in memory.

1. **Fetch**: In this phase, the CPU retrieves the instruction from memory. The address of the instruction is stored in the Program Counter (PC). The instruction is then loaded into the Instruction Register (IR).
2. **Decode**: The Control Unit interprets the instruction stored in the IR. It identifies the operation to be performed and the operands involved. This phase prepares the CPU for execution by setting up necessary control signals.
3. **Execute**: The CPU performs the operation specified by the instruction. This could be arithmetic, logic, data transfer, or control operations. The result is stored in a register or memory location.

4. **Interrupt**: If an interrupt signal is received (e.g., from an I/O device), the CPU temporarily halts the current instruction cycle. It saves the current state and services the interrupt. After handling, it resumes the instruction cycle.

Examples and Diagrams

- **Memory Reference Instruction**:
  Example: ADD M
    - Fetch: Get the instruction ADD M from memory.
    - Decode: Identify it as an addition operation with operand M.
    - Execute: Retrieve data from memory location M and add it to the accumulator.
- **I/O Instruction**:
  Example: IN
- Fetch: Load the IN instruction.
- Decode: Recognize it as an input operation.
- Execute: Read data from an input device and store it in a register.

Diagram Overview

[Memory] → [Fetch] → [Instruction Register] → [Decode] → [Control Signals] → [Execute] → [Result]

In summary, the instruction cycle is a repetitive sequence that enables the CPU to process instructions efficiently. Each phase plays a critical role in ensuring accurate and timely execution of tasks.

Question 5:

Answer:

Hardwired vs Microprogrammed Control Units

The control unit is a vital part of the CPU that directs the flow of data and instructions within the system. It interprets instructions and generates control signals to coordinate the operations of other components. Control units are classified into two types: **Hardwired** and **Microprogrammed**.

A **Hardwired Control Unit** uses fixed logic circuits to generate control signals. It is built using combinational logic and flip-flops, making it fast and efficient. The control logic is embedded directly into the hardware, which means changes require physical modifications.

A **Microprogrammed Control Unit**, on the other hand, uses a set of instructions called microinstructions stored in a control memory. These microinstructions define the control signals for each operation. The control unit reads and executes these microinstructions sequentially, making it more flexible and easier to modify.

Advantages and Disadvantages

- **Hardwired Control Unit**
    - **Advantages:**
        - Faster execution due to direct signal generation
        - Efficient for simple and repetitive tasks
    - **Disadvantages:**
    - Difficult to modify or upgrade
    - Complex to design for large instruction sets
- **Microprogrammed Control Unit**
- **Advantages:**
    - Easier to implement complex instruction sets
    - More flexible and easier to update
- **Disadvantages:**
- Slower than hardwired units due to instruction fetching
- Requires additional memory for microinstructions

Typical Applications

Hardwired control units are commonly used in **RISC (Reduced Instruction Set Computing)** architectures, where speed and simplicity are prioritized. Microprogrammed control units are preferred in **CISC (Complex Instruction Set Computing)** systems, such as older Intel processors, where flexibility and support for complex instructions are needed.

In summary, the choice between hardwired and microprogrammed control depends on the system's design goals—speed vs flexibility. Both play crucial roles in shaping the behavior and performance of modern processors.

Question 6:

Answer:

Understanding the I/O Interface and Its Role in Device Communication

The I/O interface is a hardware and software component that facilitates communication between the CPU and peripheral devices. It acts as a mediator, allowing the processor to send and receive data from external hardware like keyboards, monitors, printers, and storage devices.

The structure of an I/O interface includes:

- **Data Register**: Used to hold the data being transferred.

- **Control Register**: Manages the operation of the device (e.g., read/write).
- **Status Register**: Reports the device's condition (e.g., ready, busy).
- **Address Decoder**: Determines which device is being accessed.
- **Buffer Memory**: Temporarily stores data to handle speed differences between CPU and device.

I/O interfaces support different data transfer methods:

- **Programmed I/O**: The CPU controls the entire process, checking device status and transferring data.
- **Interrupt-driven I/O**: Devices notify the CPU when they are ready, improving efficiency.
- **DMA (Direct Memory Access)**: Allows devices to transfer data directly to memory, bypassing the CPU.

How Peripheral Devices Connect to CPU and System Bus

Peripheral devices connect to the CPU through the **system bus**, which includes:

- **Data Bus**: Transfers data between components.
- **Address Bus**: Specifies the location of data or device.
- **Control Bus**: Sends control signals to coordinate operations.

The I/O interface ensures compatibility between the CPU and devices. It converts high-level CPU commands into device-level instructions. For example, when saving a file, the CPU sends data to a hard drive via the I/O interface, which manages the actual writing process.

In conclusion, the I/O interface is essential for enabling communication between the processor and external devices. Its structured design and support for multiple transfer modes make it a key element in computer architecture.